Universidad Tecnológica de El Salvador

PROGRAMACIÓN III

Entrega 2

Marco teórico y diseño metodológico

Estado del arte, métricas y plan experimental verificable

TEMA DE INVESTIGACIÓN

Vitalis SV: prevención de citas duplicadas y conflictos de horario

Comparación experimental de estrategias de validación en una aplicación de escritorio con Java SE 21 y Microsoft SQL Server 2022

Nombre del docente: Ingeniero Eliseo Ernesto Hernández Díaz

Estudiantes	Carnet
Germán Alejandro Guzmán Dimas	17-2191-2023
Wendy Esmeralda Contreras Ardón	27-2319-2025
Denis Josué Beltrán Alvarado	25-1681-2023
José Fernando Díaz Del Cid	27-0088-2025
Gerson Mauricio Nataren Rodríguez	27-0567-2025
Joosue Daniel Martínez Rosales	25-4699-2024

Grupo 6 · Octubre de 2026

Repositorio: https://github.com/GermanAleGD/GRUPO-6_Progra3
Etiqueta de la entrega: `v0.2-marco-metodologia`

1. Planteamiento de la investigación

Vitalis SV es un prototipo de aplicación de escritorio para administrar pacientes, médicos y citas, desarrollado con Java SE 21, Swing y Microsoft SQL Server 2022 (repositorio del grupo, delimitación tecnológica). Una de sus reglas críticas es evitar las citas duplicadas y los conflictos de horario. Esa regla puede colocarse en la capa de aplicación (se consulta y luego se inserta) o en la propia base de datos (restricción única), y la decisión afecta tanto la integridad de la agenda como el tiempo de respuesta percibido por quien registra la cita.

La literatura indica que las validaciones hechas solo en la aplicación pueden fallar cuando hay accesos concurrentes (Bailis et al., 2015; Cheng et al., 2023), pero esa evidencia proviene de aplicaciones web con ORM y no de aplicaciones de escritorio con JDBC y SQL Server, ni del dominio de citas médicas. De ahí surge el problema que se investiga.

Pregunta de investigación. ¿Qué estrategia de validación de citas (en la aplicación, con restricción única en SQL Server, híbrida, o en la aplicación con aislamiento SERIALIZABLE) ofrece el mejor equilibrio entre integridad de los datos y tiempo de respuesta en Vitalis SV cuando varios usuarios intentan reservar la misma franja de forma concurrente?

1.1 Objetivo general

Comparar experimentalmente cuatro estrategias de validación de citas médicas (A1 a A4) en Vitalis SV, en términos de integridad de los datos, tiempo de respuesta, consumo de memoria y mantenibilidad, para fundamentar la elección de la estrategia del prototipo.

1.2 Objetivos específicos

N.º	Objetivo específico	Evidencia verificable
OE1	Implementar las cuatro estrategias detrás de una interfaz común, con pruebas automatizadas.	Código en el repositorio, `mvn test` y flujo de integración continua en verde.
OE2	Medir citas duplicadas aceptadas, tiempo de respuesta (p50 y p95), memoria pico e interbloqueos por combinación de estrategia, concurrencia y volumen.	Archivos `resultados/*.csv` con 30 iteraciones por combinación.
OE3	Comparar estadísticamente las estrategias con tamaño del efecto e intervalos de confianza del 95 %.	Script de análisis versionado y tablas de resultados.
OE4	Evaluar la mantenibilidad de cada estrategia mediante complejidad ciclomática y cobertura de pruebas.	Informes de SonarQube/PMD y JaCoCo.
OE5	Documentar el entorno y el procedimiento para que otra persona pueda replicar las mediciones.	`docs/entorno.md`, scripts SQL y registros de decisiones (ADR).

Tabla 0. Objetivos específicos y su evidencia verificable.

1.3 Hipótesis

H1: la estrategia solo en aplicación (A1) aceptará al menos una cita duplicada en alguna corrida con 4 o 16 hilos, mientras que A2, A3 y A4 aceptarán cero. **H2:** A2 tendrá un tiempo de respuesta p50 menor o igual que A3 (una sentencia frente a dos) y menor que A4. **H3:** A4 mostrará la mayor tasa de interbloqueos con 16 hilos.

1.4 Justificación

Justificación técnica. Una restricción de unicidad que falla bajo concurrencia produce citas duplicadas y deja la agenda en un estado inconsistente. La literatura muestra que muchas aplicaciones resuelven la integridad con validaciones en la aplicación y aislamiento débil (Bailis et al., 2015; Tang et al., 2022; Cheng et al., 2023), pero no cuantifica el costo de las alternativas en una aplicación de escritorio con JDBC y SQL Server. Los resultados darán datos objetivos para decidir la arquitectura de Vitalis SV y quedarán registrados en un ADR.

Justificación social y sanitaria. La inasistencia a consulta externa ronda el 23 % entre especialidades (Dantas et al., 2018) y las mejoras en el sistema de agenda se asocian con menos inasistencias (Mazaheri Habibi et al., 2024). Una agenda sin duplicados ni conflictos de horario es la base sobre la que se puede construir ese tipo de mejoras. Además, al tratarse de información de salud, el proyecto adopta desde el diseño los criterios de confidencialidad, integridad y disponibilidad (Cureus, 2022) y trabaja solo con datos ficticios, en coherencia con la Ley para la Protección de Datos Personales (Asamblea Legislativa, 2024).

Justificación académica. El estudio integra contenidos de Programación III (programación orientada a objetos, acceso a datos con JDBC, pruebas y control de versiones) con el método de la ingeniería de software empírica: factores, réplicas, métricas definidas y análisis estadístico (Wohlin et al., 2012). Aporta además un procedimiento reproducible que otros grupos podrán reutilizar.

Viabilidad. Las herramientas son gratuitas o de licencia de desarrollo (Java, SQL Server Developer, GitHub), todo corre en un solo equipo y los datos se generan con un script propio, por lo que no hay costos de infraestructura ni dependencias externas.

1.5 Alcance y limitaciones

Incluye	No incluye
Aplicación de escritorio con Java SE 21 y Swing, y base de datos Microsoft SQL Server 2022 en el mismo equipo.	Aplicaciones móviles ni servicios en la nube.
Gestión de pacientes, médicos y citas, con registro de los medicamentos indicados por el médico.	Conexión con los sistemas reales del Instituto Salvadoreño del Seguro Social.
Validación de citas duplicadas y de conflictos de horario (RN1 y RN2).	Datos reales de pacientes; todas las pruebas usan datos ficticios.
Experimento con cuatro estrategias, tres niveles de concurrencia y dos volúmenes de datos.	Otros motores de base de datos, bases remotas con latencia de red y pruebas de usabilidad con usuarios.

Tabla 0b. Alcance. Limitaciones principales: un solo equipo, usuarios simulados con hilos de una misma JVM y resultados válidos para el entorno descrito en 4.4.

2. Estado del arte

2.1 Búsqueda documentada

La búsqueda se realizó el 16 de octubre de 2026 con un motor de búsqueda web general, revisando hasta los 10 primeros resultados por consulta, y se complementó con documentación oficial y obras metodológicas de referencia. El registro completo, con enlaces, se conserva en `docs/estado-del-arte.md`.

N.º	Cadena de búsqueda	Revisados	Selecc.
1	hospital appointment scheduling system design double booking prevention database	10	0
2	JDBC PreparedStatement performance evaluation SQL injection empirical study	10	0
3	outpatient appointment scheduling systematic review no-show management information system	10	2
4	Java microbenchmark JMH methodology empirical study performance testing	10	1
5	SQL Server unique index constraint concurrency isolation level performance study	10	0
6	electronic health record confidentiality integrity availability security healthcare information systems review	10	1
7	Java Swing desktop application MVC layered architecture maintainability study	10	0
8	feral concurrency control application-level validation vs database constraints integrity empirical study	10	1
9	Ley de Protección de Datos Personales El Salvador aprobada Asamblea Legislativa	10	1
10	database concurrency anomalies in web applications isolation level bugs study 2022 2023	10	2

Tabla 1. Registro de búsquedas. Total revisado: 100 resultados; seleccionados: 8. A estos se suman 7 fuentes incorporadas directamente por ser documentación oficial, estándares u obras metodológicas (15 fuentes en total).

2.2 Criterios de inclusión y exclusión

Criterios de inclusión	Criterios de exclusión
Publicado entre 2019 y 2026. Excepción justificada: obras fundacionales o metodológicas y estudios empíricos directamente relacionados con el problema.	Sin metodología descrita o sin datos verificables.
Con datos empíricos, revisión sistemática, estándar o documentación oficial.	Fuente comercial, blog, apuntes o diapositivas sin datos verificables (por ejemplo, resultados de Scribd y blogs técnicos).
Relación directa con citas médicas, integridad y concurrencia, JDBC/SQL Server, seguridad de datos de salud o medición en Java.	Duplicado de otro estudio ya incluido, o propuesta de herramienta sin evaluación.

2.3 Matriz comparativa de antecedentes

Autor y año	Objetivo	Método y entorno	Hallazgo principal	Vacío detectado
Bailis et al. (2015)	Evaluar si los mecanismos de integridad en la capa de aplicación protegen los datos.	Análisis empírico de aplicaciones de código abierto con ORM (Ruby on Rails) y experimentos de corrupción por concurrencia.	Las validaciones en la aplicación son más de 37 veces más usadas que las transacciones; hasta 86,9 % garantizan integridad y el resto puede corromper datos.	Solo aplicaciones web con ORM; no mide tiempo de respuesta ni cubre JDBC, SQL Server ni citas médicas.
Tang et al. (2022)	Entender cómo implementan los desarrolladores transacciones propias (ad hoc) en aplicaciones web.	Estudio de aplicaciones web de código abierto.	Los desarrolladores coordinan la concurrencia en la aplicación con mecanismos propios, con riesgos de corrección.	Dominio web; no compara costo frente a una restricción declarativa de la base de datos.
Cheng et al. (2023)	Determinar quién debe controlar la concurrencia: el desarrollador o la base de datos.	Análisis de 93 errores reales de concurrencia en aplicaciones con bases de datos.	Se usa con frecuencia un aislamiento débil (READ COMMITTED) y los desarrolladores corrigen con soluciones ad hoc complejas y a veces incorrectas.	No cuantifica el costo de cada alternativa de prevención en un motor concreto ni en una aplicación de escritorio.
Costa et al. (2021)	Identificar malas prácticas en benchmarks JMH.	123 sistemas Java de código abierto; análisis estático y experimentos de caso.	Cinco malas prácticas son frecuentes y alteran significativamente los resultados.	No aborda mediciones con base de datos externa ni concurrencia con JDBC.
Mazaheri Habibi et al. (2024)	Evaluar el efecto del acceso abierto en la tasa de inasistencia a consulta externa.	Revisión sistemática: 16 artículos seleccionados de 23 403.	En 10 de 16 estudios (62,5 %) la inasistencia bajó de forma significativa, con condiciones como análisis de necesidades.	Se centra en inasistencia; no examina la integridad técnica de la agenda.
Cureus (2022)	Revisar seguridad y privacidad de los registros electrónicos de salud.	Revisión narrativa de literatura (marco HIPAA, confidencialidad y disponibilidad).	Privacidad, seguridad y confidencialidad son los aspectos más importantes de los sistemas de registros de salud.	Sin evidencia experimental sobre validaciones de integridad en aplicaciones pequeñas ni contexto salvadoreño.

Tabla 2. Matriz de antecedentes. La última columna fundamenta la justificación de este estudio.

2.4 Síntesis y vacío detectado

Bailis et al. (2015) mostraron que, al delegar la integridad en validaciones de la aplicación, las restricciones de unicidad pueden violarse bajo concurrencia, mientras que el mecanismo equivalente de la base de datos sí la garantiza. Tang et al. (2022) y Cheng et al. (2023) confirman, con corpus más recientes, que la coordinación ad hoc en la aplicación es habitual y propensa a errores, y que el aislamiento por defecto suele ser débil. Costa et al. (2021) advierten que medir el desempeño en Java exige cuidado metodológico, lo que condiciona el diseño de este trabajo. En el dominio de la salud, Mazaheri Habibi et al. (2024) y la revisión publicada en Cureus (2022) se ocupan de la inasistencia y de la seguridad de la información, pero no de la integridad de la agenda.

Vacío. No se encontró un estudio que compare, con un diseño reproducible, estrategias de validación de citas en una aplicación de escritorio Java con JDBC y SQL Server midiendo a la vez integridad, tiempo de respuesta y mantenibilidad. Este trabajo busca cubrir ese vacío a pequeña escala (prototipo, datos ficticios, un solo equipo).

3. Marco teórico

3.1 Sistemas de información en salud y gestión de citas

Un sistema de información recopila, procesa, almacena y entrega información para apoyar las actividades de una organización. En salud, la gestión de citas organiza la atención según la disponibilidad de los médicos. La inasistencia es un problema persistente: una revisión de 105 estudios reporta un promedio cercano al 23 % entre especialidades (Dantas et al., 2018), y los sistemas de agenda diseñados según las necesidades reales de pacientes y proveedores se asocian con menos inasistencias (Mazaheri Habibi et al., 2024). Además, los registros de salud exigen confidencialidad, integridad y disponibilidad (Cureus, 2022). En El Salvador, la Ley para la Protección de Datos Personales (Asamblea Legislativa, 2024) exige consentimiento expreso y por escrito para datos sensibles y la notificación de vulneraciones; por ello Vitalis SV se prueba solo con datos ficticios.

Aporte: *de aquí se derivan las reglas de negocio que se verificarán: RN1, un médico no puede tener dos citas activas en la misma franja; RN2, un paciente no puede tener dos citas activas en la misma franja.*

3.2 Integridad de datos y control de concurrencia

Las transacciones deben cumplir las propiedades ACID; la “I” (aislamiento) se controla con niveles de aislamiento. SQL Server usa READ COMMITTED por defecto, que evita lecturas sucias pero no impide que dos sesiones lean que una franja está libre y luego inserten ambas (condición de carrera de tipo comprobar-y-actuar). SERIALIZABLE evita ese problema mediante bloqueos de rango, a costa de más bloqueos y posibles interbloqueos (Microsoft, 2022). Una restricción única (índice único filtrado) delega la garantía al motor de forma atómica. La evidencia indica que muchas aplicaciones operan con aislamiento débil y resuelven la concurrencia con código propio (Bailis et al., 2015; Tang et al., 2022; Cheng et al., 2023).

Aporte: *define los cuatro niveles del factor A: A1, validación en aplicación con READ COMMITTED; A2, solo restricción única en SQL Server; A3, híbrida (consulta previa más restricción única); A4, validación en aplicación con transacción SERIALIZABLE.*

3.3 Acceso a datos con JDBC, seguridad y arquitectura

JDBC es la interfaz estándar de Java para bases de datos relacionales; Vitalis SV usa el Microsoft JDBC Driver for SQL Server (Microsoft, 2025). Las sentencias parametrizadas (`PreparedStatement`) separan el código SQL de los datos del usuario (Oracle, 2023) y son la defensa principal contra la inyección SQL, categoría A03:2021 del OWASP Top 10 (OWASP Foundation, 2021). La aplicación separa interfaz (Swing), validaciones y acceso a datos, y modela pacientes, médicos y citas como objetos con encapsulamiento.

Aporte: *las cuatro estrategias usarán sentencias parametrizadas y la misma capa de acceso a datos, de modo que la seguridad frente a inyección SQL sea una constante y no una variable de confusión.*

3.4 Calidad del software y medición experimental

ISO/IEC 25010:2023 define el modelo de calidad del producto (eficiencia de desempeño, fiabilidad, seguridad, mantenibilidad, entre otras). Para medir tiempos en la JVM se usa JMH, que controla el calentamiento y los efectos del compilador JIT; sin embargo, Costa et al. (2021) documentan que las malas prácticas al escribir benchmarks distorsionan los resultados, por lo que se seguirán sus recomendaciones. El diseño de experimentos (factores, niveles, réplicas, aleatorización) sigue a Wohlin et al. (2012), y el registro de la búsqueda bibliográfica sigue las pautas de Kitchenham y Charters (2007). También se toman como referencia IEEE 829 (documentación

de pruebas) y SWEBOK (delimitación de áreas de conocimiento).

***Aporte:** justifica las métricas y su anclaje a un estándar, el uso de JMH y la estructura del experimento.*

3.5 Operacionalización de variables

Variable	Definición operacional	Unidad	Instrumento	ISO/IEC 25010
V1. Citas duplicadas aceptadas	Número de pares (médico, fecha y hora) con más de una cita activa al terminar cada iteración, contado con una consulta GROUP BY ... HAVING COUNT(*) > 1.	citas/iteración	Script `verificar_integridad.sql`	Seguridad (integridad)
V2. Tiempo de respuesta	Milisegundos entre el inicio de la primera sentencia del intento de reserva y la recepción de la confirmación o del rechazo. Se reportan p50 y p95 por iteración.	ms	JMH, modo SampleTime	Eficiencia de desempeño
V3. Memoria pico	Máximo de memoria del montículo en uso durante la iteración, muestreado cada 100 ms.	MB	MemoryMXBean y `-Xlog:gc`	Eficiencia de desempeño
V4. Tasa de interbloqueos	Intentos terminados con el error 1205 de SQL Server, entre el total de intentos.	%	`SQLException.getErrorCode()`	Fiabilidad
V5. Complejidad ciclomática	Caminos independientes por método de validación.	entero	SonarQube, PMD	Mantenibilidad
V6. Cobertura de pruebas	Porcentaje de líneas ejecutadas por la suite de pruebas en el módulo de validación.	%	JaCoCo	Mantenibilidad

Tabla 3. Operacionalización de variables.

4. Diseño metodológico

4.1 Tipo y enfoque

Estudio cuantitativo de tipo experimental con benchmarking comparativo: se manipulan dos factores técnicos en un entorno controlado, se miden variables de respuesta con repeticiones y se complementa con medición estática de la calidad del código. Se eligió este diseño porque la pregunta compara tratamientos técnicos y exige variables controladas, repeticiones y entorno estable.

4.2 Diseño del experimento

Elemento	Declaración
Factor A: estrategia de validación (4 niveles)	A1: consulta y luego inserción en la aplicación, READ COMMITTED. A2: inserción directa y captura del error de unicidad (2601/2627). A3: consulta previa más inserción, con índice único como respaldo. A4: consulta e inserción en transacción SERIALIZABLE.
Factor B: concurrencia (3 niveles)	1, 4 y 16 hilos, cada uno con su propia conexión JDBC.
Factor C: volumen de citas existentes (2 niveles)	10.000 y 100.000 citas históricas precargadas.
Combinaciones	4 x 3 x 2 = 24 combinaciones.
Variables de respuesta	V1 a V4 por iteración (V5 y V6 se miden una vez por estrategia sobre el código).
Repeticiones	30 iteraciones de 10 s por combinación, precedidas de 5 iteraciones de calentamiento descartadas; 1 fork de JMH.
Carga de trabajo	Cada hilo intenta reservas continuamente sobre un conjunto de 10.000 franjas (500 médicos x 20 franjas). El 20 % de los intentos apunta a 50 franjas “calientes” para provocar competencia; el resto se distribuye de forma uniforme. Una franja ya ocupada debe ser rechazada.
Control (constantes)	Misma máquina dedicada, sin otras cargas; equipo conectado a corriente y plan de energía de alto rendimiento; actualizaciones y antivirus pausados; mismo esquema; índice no único en A1 y A4 sobre las mismas columnas que el índice único de A2 y A3 (para aislar el efecto de la unicidad y no el del índice); mismo montículo (`-Xms1g -Xmx1g`); mismo código de acceso a datos.

Elemento	Declaración
Orden y semilla	Las 24 combinaciones se ejecutan en orden aleatorio con semilla fija (seed = 42); la selección de franjas usa Random(42 + índice de hilo).

Tabla 4. Diseño factorial. Duración estimada: 24 x 35 x 10 s = 8.400 s (unas 2,5 a 3 horas con preparación).

4.3 Procedimiento ejecutable

1. Restaurar la base `vitalis_bench` con `01_schema.sql` y `02_seed.sql` (datos sintéticos, seed = 42).
2. Crear el índice según la estrategia (`03_indice_no_unico.sql` para A1 y A4; `03_indice_unico_filtrado.sql` para A2 y A3).
3. Registrar versiones y hardware en `docs/entorno.md` (modelo de CPU, RAM, SSD, compilación exacta de Windows).
4. Entre combinaciones: `CHECKPOINT; DBCC DROPCLEANBUFFERS;` y limpiar las citas de prueba de la iteración anterior (marcadas con origen = 'bench').
5. Ejecutar el orquestador, que baraja las 24 combinaciones con seed = 42 y lanza JMH con `-p estrategia=... -p volumen=...` y `@Threads` según el factor B.
6. Al cerrar cada iteración, ejecutar `verificar_integridad.sql` y guardar V1 a V4 en `resultados/*.csv`.
7. Medir V5 y V6 con SonarQube/PMD y JaCoCo sobre el módulo de validación.
8. Analizar los CSV con el script de R o Python versionado en `analisis/` (sección 4.6) y archivar los resultados con la etiqueta del commit.

4.4 Entorno experimental

Componente	Especificación (de la delimitación tecnológica del repositorio)
Sistema operativo	Windows 11 Home 24H2, 64 bits (compilación exacta por registrar antes de medir).
Hardware mínimo de referencia	Procesador x64 compatible con Windows 11, 8 GB de RAM, 20 GB libres, SSD recomendado (modelo exacto por registrar).
Java	Oracle JDK 21.0.12+7 (Java SE 21); interfaz Swing; opciones de la JVM `-Xms1g -Xmx1g`.
Base de datos	Microsoft SQL Server 2022 Developer, CU27, compilación 16.0.4295.3; aplicación y base en el mismo equipo.
Conector	Microsoft JDBC Driver for SQL Server 12.10.2.
Medición y calidad	JMH 1.37; JUnit; JaCoCo; SonarQube Community; PMD (versiones fijadas en `pom.xml`).
Parámetros modificados	SQL Server: max server memory = 4.096 MB, modelo de recuperación SIMPLE para la base de pruebas, MAXDOP por defecto. Aislamiento READ COMMITTED (SQL Server por defecto) salvo en A4.
Reproducibilidad	Scripts SQL y configuración en el repositorio. Para la integración continua se usará un contenedor oficial de SQL Server 2022 en `ubuntu-latest` con las pruebas de la lógica sin interfaz gráfica.

4.5 Población, muestra o corpus de datos

No participan personas. El corpus es sintético: 500 médicos, 20.000 pacientes y las citas históricas (10.000 o 100.000 según el factor C) generadas por un script propio con semilla 42, en un rango de fechas distinto al de las 10.000 franjas de prueba para que no haya colisiones previas. La regla es defendible porque reproduce el volumen de una clínica mediana y permite repetir la generación exacta.

4.6 Plan de análisis

Se calcularán media, mediana, desviación estándar y diagramas de caja por combinación. Se verificará normalidad con Shapiro-Wilk; si se cumple se aplicará ANOVA factorial (A x B x C) y, si no, Kruskal-Wallis con pruebas por pares de Mann-Whitney y corrección de Holm. Se reportarán el tamaño del efecto (delta de Cliff

o eta cuadrado) e intervalos de confianza del 95 % por bootstrap. Para V1 se reportará la proporción de iteraciones con al menos un duplicado, con intervalo de Clopper-Pearson. Nivel de significancia: $\alpha = 0{,}05$. Se recuerda que correlación no implica causalidad y que los resultados se limitan al entorno descrito.

4.7 Amenazas a la validez

Tipo	Riesgo	Mitigación
Interna	Otro proceso (antivirus, actualizaciones) consume CPU o disco durante la medición.	Máquina dedicada, servicios pausados, 30 repeticiones y descarte de atípicos solo con criterio documentado.
Interna	El estado de la base cambia entre iteraciones.	Limpieza de citas de prueba y reinicio de cachés entre combinaciones; verificación de V1 al cerrar cada iteración.
Externa	Los resultados aplican solo a SQL Server 2022 y a un equipo con 8 GB de RAM.	Declarar el alcance; replicar en un segundo equipo si es viable.
Externa	Los hilos simulan usuarios concurrentes desde una sola JVM y la base está en el mismo equipo (sin latencia de red).	Declararlo como límite; en futuras entregas, probar con una base remota.
De constructo	Las franjas calientes pueden sobrerrepresentar la competencia real.	Reportar también la corrida con carga uniforme y justificar el 20 % con la literatura de agendamiento.
De conclusión	Pocas repeticiones para afirmar una diferencia.	30 iteraciones por combinación; reportar dispersión, tamaño del efecto e intervalos.

4.8 Consideraciones éticas

No intervienen personas ni datos reales: se usan datos ficticios y no hay conexión con sistemas del Instituto Salvadoreño del Seguro Social. Se respetan las licencias: SQL Server Developer es para desarrollo y pruebas, y se revisarán los términos aplicables del JDK usado (OpenJDK como alternativa). **Declaración de uso de inteligencia artificial:** se empleó el asistente Claude (Anthropic) para apoyar la búsqueda inicial de fuentes, proponer la estructura del diseño experimental y redactar y maquetar el borrador de este documento. Las referencias (DOI y enlaces) y las decisiones de diseño deben ser verificadas por el grupo, que es responsable del contenido final.

5. Avance técnico y plan del repositorio

Al 19 de septiembre de 2026 el repositorio contiene la delimitación teórica, tecnológica y expresiva y la tabla de viabilidad, con commits de los seis integrantes. Para esta entrega el repositorio debe pasar a contener software que compile y pruebas que pasen, con la siguiente estructura:

Elemento	Contenido previsto en el repositorio
Esqueleto del prototipo	Proyecto Maven (Java 21) que compila con `mvn test`: interfaz `EstrategiaValidacion`, clases A1 a A4 y al menos una prueba JUnit que pase.
Dependencias declaradas	`pom.xml` con versiones fijadas: mssql-jdbc 12.10.2, JMH 1.37, JUnit y JaCoCo.
Entorno reproducible	`docs/entorno.md` y scripts en `sql/`.
Integración continua	`.github/workflows/ci.yml`: compila y ejecuta pruebas en cada push.
Decisiones registradas	`docs/adr/0001-estrategias-de-validacion-de-citas.md` y `0002-jmh-como-herramienta-de-medicion.md`.
Fichas de lectura	`docs/estado-del-arte.md` con la matriz y las cadenas de búsqueda (sección 2).
Convenciones de trabajo	`docs/CONTRIBUTING.md`: commits en inglés con Conventional Commits (conventionalcommits.org), plantilla `.gitmessage`, ramas por tarea y pull requests; no se trabaja directamente en `main`.
Tablero y etiqueta	Issues por objetivo específico (OE1 a OE5) con responsables, y etiqueta `v0.2-marco-metodologia`.

Referencias

Asamblea Legislativa de El Salvador. (2024). *Ley para la Protección de Datos Personales*. https://www.icnl.org/wp-content/uploads/Ley-para-la-proteccion-de-datos-personales.pdf

Bailis, P., Fekete, A., Franklin, M. J., Ghodsi, A., Hellerstein, J. M., & Stoica, I. (2015). Feral concurrency control: An empirical investigation of modern application integrity. *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. https://amplab.cs.berkeley.edu/publication/feral-concurrency-control-an-empirical-investigation-of-modern-application-integrity

Cheng, C., Han, M., Xu, N., & Blanas, S. (2023). Developer's responsibility or database's responsibility? Rethinking concurrency control in databases. *13th Annual Conference on Innovative Data Systems Research (CIDR '23)*. https://www.vldb.org/cidrdb/2023/developers-responsibility-or-databases-responsibility-rethinking-concurrency-control-in-databases.html

Costa, D. E., Bezemer, C.-P., Leitner, P., & Andrzejak, A. (2021). What's wrong with my benchmark results? Studying bad practices in JMH benchmarks. *IEEE Transactions on Software Engineering*. https://research.chalmers.se/en/publication/511625

Dantas, L. F., Fleck, J. L., Cyrino Oliveira, F. L., & Hamacher, S. (2018). No-shows in appointment scheduling: A systematic literature review. *Health Policy, 122*(4), 412–421. https://doi.org/10.1016/j.healthpol.2018.02.002

Health records database and inherent security concerns: A review of the literature. (2022). *Cureus*. https://doi.org/10.7759/cureus.30168

ISO/IEC. (2023). *ISO/IEC 25010:2023 Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model*. International Organization for Standardization.

Kitchenham, B., & Charters, S. (2007). *Guidelines for performing systematic literature reviews in software engineering* (EBSE Technical Report EBSE-2007-01). Keele University.

Mazaheri Habibi, M. R., Mohammad Abadi, F., Tabesh, H., Vakili-Arki, H., Abu-Hanna, A., Ghaddaripouri, K., & Eslami, S. (2024). Evaluation of no-show rate in outpatient clinics with open access scheduling system: A systematic review. *Health Science Reports, 7*(7). https://doi.org/10.1002/hsr2.2160

Microsoft. (2022). *Transaction locking and row versioning guide* [Documentación de SQL Server 2022]. https://learn.microsoft.com/en-us/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide

Microsoft. (2025). *Microsoft JDBC Driver for SQL Server* (versión 12.10) [Documentación]. https://learn.microsoft.com/en-us/sql/connect/jdbc/microsoft-jdbc-driver-for-sql-server

Oracle. (2023). *Java SE 21 API: java.sql.PreparedStatement*. https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/PreparedStatement.html

OWASP Foundation. (2021). *OWASP Top 10:2021 — A03 Injection*. https://owasp.org/Top10/

Tang, C., Wang, Z., Zhang, X., Yu, Q., Zang, B., Guan, H., & Chen, H. (2022). Ad hoc transactions in web applications: The good, the bad, and the ugly. *Proceedings of the 2022 International Conference on Management of Data (SIGMOD '22)*. ACM.

Wohlin, C., Runeson, P., Höst, M., Ohlsson, M. C., Regnell, B., & Wesslén, A. (2012). *Experimentation in software engineering*. Springer.

De las 15 fuentes, 11 (73 %) corresponden a 2021–2026; las 4 anteriores son estudios fundacionales o guías metodológicas (Bailis et al., 2015; Dantas et al., 2018; Kitchenham y Charters, 2007; Wohlin et al., 2012). Todas están citadas en el texto.